Libraries

For SWIFT Translator

User Guide

This document explains the details of the SWIFT Standards Libraries, and how they can be used by SWIFT Translator standalone customers and SIL/IPLA developer seat customers

21 August 2020

Table of Contents

Table of Contents **2**

Preface **3**

1 WHAT **4**

1.1 What are "Libraries"? 4
1.2 Content and Use 5
1.2.1 Delivery and Content of the SWIFT Libraries 5
1.2.2 Syntax Errors vs Validation Errors 5
1.2.3 How to Use and Invoke/Upload the Library Functions 5
1.3 Who Does What? 8
1.4 Available Libraries and Documentation 8

2 Translation Library Details - Structure **10**

2.1 Base Message 10
2.2 Mapping Hierarchy 10
2.2.1 Level 1 - Header Mapping 10
2.2.2 Level 2 - Business Message Mapping 13
2.2.3 Level 3 - Decision Logic 13
2.2.4 Level 4 - Interface Mapping 14

3 HOW to Integrate the Libraries and Invoke the Functions Directly from your Application(s) **15**

3.1 MVT API 15
3.1.1 Overview 15
3.1.2 Integration – Prerequisites 15
3.1.3 Flow 16
3.1.4 Project Information – Unique Keys to Invoke a Function 17
3.1.5 The Message Parsing Function 17
3.1.6 The Validation Function 18
3.1.7 The Mapping Function 18
3.1.8 Other Functions 19
3.1.9 Runtime Parameters 20
3.1.10 Chaining Functions 21
3.2 Runtime API 22
3.3 Webservices 22
3.4 Maven 22
3.5 Command Line Tools 22

4 HOW to Customise a Library **23**

4.1 Use Cases, Types of "Customisation" 23
4.2 HOW to Customise a SWIFT Standards Library? 24
4.2.1 How to Import a Library in SWIFT Translator Designer 24
4.2.2 General Guidelines 26

5 Related Documentation **27**

Legal Notices **28**

Preface

About this document

This document describes the generic features and functioning of SWIFT Translator libraries. Library-specific details are out of the scope of this document.

These libraries can be directly integrated and used within your applications. They can also be customised first (in the SWIFT Translator Designer) before you actually start validating and/or translating your messages.

For both scenarios, this document describes how to integrate and use the libraries. In that respect, it also includes references to more technical documents, targeted to your developers to guide them in setting up and integrating the libraries. These technical documents are available on the Knowledge Centre and the Download Centre.

Please read this document before you start integrating and using the Standards libraries that you have purchased.

Intended audience

This document is for the following audience:

- SWIFT Translator Standalone customers, having a Runtime only license.

  They can use the Standards libraries, as delivered, to validate and/or translate messages in their environment.
- SWIFT Translator Standalone customers, having a "bundle" licence: both the Designer and the Runtime component.

  Before actually validating or translating messages with the delivered libraries, they can customise the included definitions and rules first in the Designer (GUI).
- SIL Developer Seat Licence customers.
  They can either use the Standards libraries as delivered, or customise themselves the included definitions and rules in the SWIFT Translator Designer (GUI).
- IPLA Developer Seat Licence customers.
  They can either use the Standards libraries as delivered, or customise themselves the included definitions and rules in the SWIFT Translator Designer (GUI).

First edition

This is the first edition of the document.

1 WHAT

1.1 What are "Libraries"?

Libraries are predefined definitions of messages and mappings. Such libraries allow you, for example, to validate your messages against the ISO 20022 format or to translate your current MT formatted messages to the new ISO 20022 MX formatted messages, and vice versa.

SWIFT provides pre-built libraries for many industry standard messages, and continues to extend its offer in function of the demands in the market. These demands are mainly, but not only, linked to the ISO 20022 migration timelines of market infrastructures migrations to ISO 20022 all over the globe.

See Available Libraries and Documentation for SWIFT current offer. If you need a library that is not yet in the list, or if you need more information about libraries, please contact your account manager.

These libraries are inherently linked to the SWIFT Translator tool. SWIFT Translator is a messaging format translation solution which allows you to define, validate, and translate messages from and to messaging formats. For more information about SWIFT Translator, see the SWIFT Translator Service Description.

The SWIFT Translator tool consists of two components: the Designer and the Runtime. The Runtime component is required to make use of the libraries. In case customisation is also required, you need the Designer, or you can contact SWIFT Professional Services for customisation.

The SWIFT Translator Runtime component can be integrated outside of a SWIFT Interface (standalone), or as part of your existing SWIFT interface (SIL, IPLA,…).

This document is specifically meant for these SWIFT Translator standalone, SIL and IPLA developer seat customers.

1.2 Content and Use

1.2.1 Delivery and Content of the SWIFT Libraries

The SWIFT Standards libraries are delivered on the Download Centre in the form of .jar files. Such .jar file contains all definitions of each relevant message, mapping, condition, etc.

In particular, the .jar file(s) consist of one or more of the following feature types:

- Message definitions: required to parse a message into an internal structural form.
- Validation rule sets: validating whether an input message contains errors.
- Mapping definitions: translating one input message into one output message as per specification.
- Conditions: checking a condition on an input message and return true or false.
- Enrichment rules sets: allowing to add data to a message.

Note *The message definition is used to parse plain text into a structural form. This structural form will be the input of the other features.*

1.2.2 Syntax Errors vs Validation Errors

A core function of a library is message validation (described further). A message validation has usually two parts: the syntax and the semantic. The syntax is defined in the message definition, while the semantic can be defined in the validation rule set. The parsing does the syntax validation and the semantic validation can be executed by applying the validation rule set containing the semantic. In both stages of the process, errors can be reported.

- Syntax errors after parsing
- Validation errors

Syntax errors are triggered because the base definition of a message type has been violated. Even when parsing a message gives a result, it is important to check for syntax errors. When syntax errors have occurred, the validation might not be trustworthy. It is therefore advised to stop the process as soon as a syntax error occurs.

1.2.3 How to Use and Invoke/Upload the Library Functions

It is important to notice that libraries are not "plug and play". They are delivered as .jar files, but they cannot be deployed like a piece of code in your target environment. You do require the SWIFT Translator Runtime component.

Depending on your situation and concrete usage of the purchased library(/ies), there are two options:

- You can use these pre-built libraries as is to validate and translate your (application's) messages against. You use them as they are delivered and you do not need to change or adapt the included definitions, rules, and mappings.

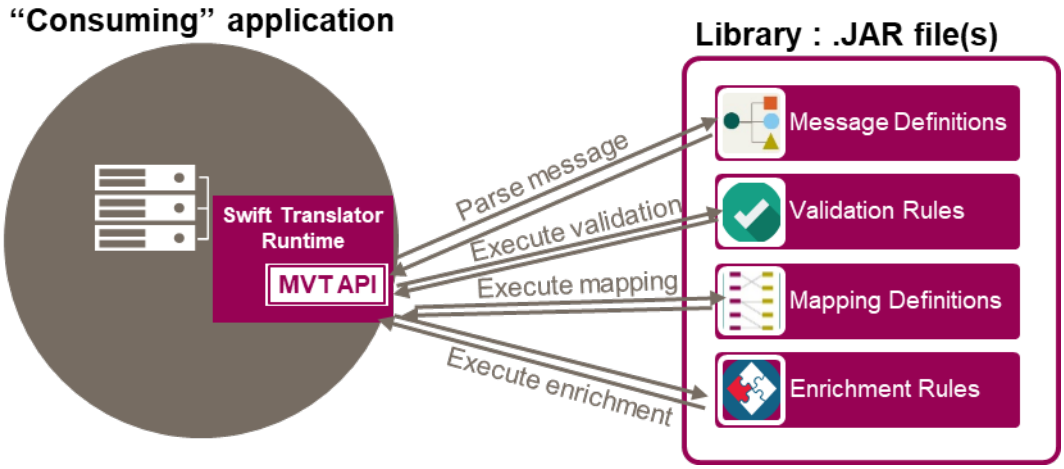
  In this case, the delivered .jar file can directly be used in your environment via the SWIFT Translator Runtime.
- You need to customise or extend the library: then you have to use the SWIFT Translator Designer component. This component allows you to "customise", that is, edit, the delivered libraries.

1.2.3.1 Direct Load in the SWIFT Translator Runtime, Use the Libraries as Delivered

In this option, you only need the SWIFT Translator Runtime standalone component on top of the library. You need to deploy this Java-based software component in the application where you want to validate and/or translate your messages. You are going to validate and/or translate the messages using the exact definitions and rules delivered within the library.

How does this work?

There are different ways to integrate the libraries in your application landscape. The most common and easy one is to use the Message Validation and Translation API (MVT API). The MVT API is provided by SWIFT and included in the Runtime Component of SWIFT Translator. It is a predefined, simplified Java API which reads the .jar file(s) and triggers its functions.



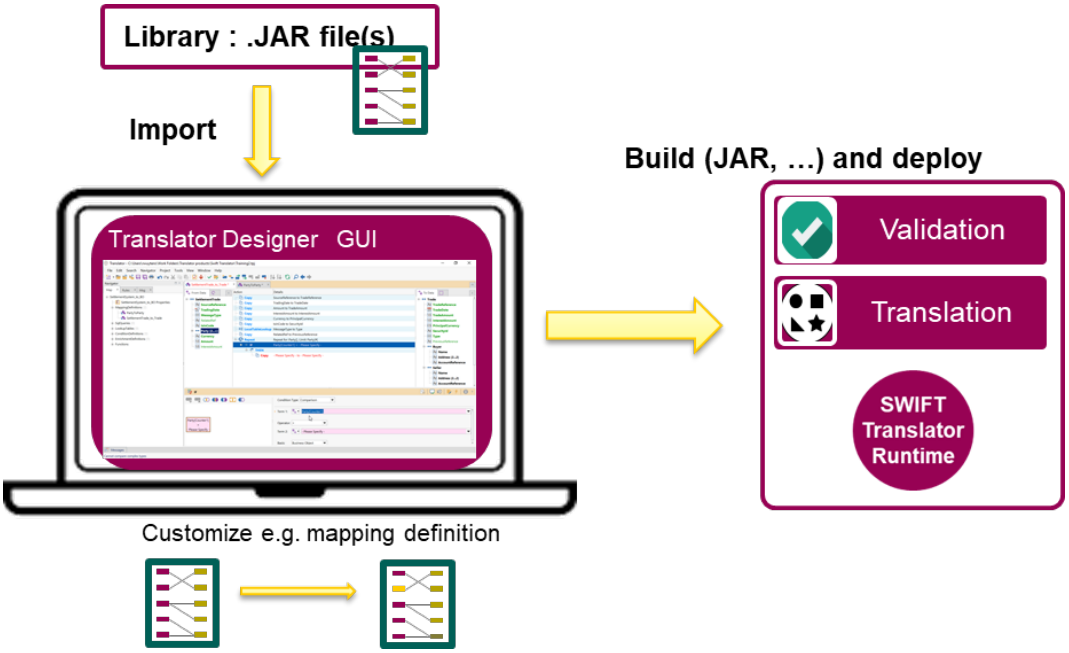
This MVT API integration is explained in detail further in this document.

Next to the MVT API, other integration options exist. See also further in the document.

1.2.3.2 Import in the SWIFT Translator Designer to Extend the Library(/ies), Before Actually Using Them

The library (that is, the .jar file downloaded from the Download Centre) can also be imported in SWIFT Translator Designer. This is necessary if you need to customise the delivered library: the SWIFT Translator Designer GUI allows the user to create additional logic around the library, such as new mappings, operations, or services.

In this case, the library can be considered as a predefined project (a project as defined in SWIFT Translator Designer) containing definitions of messages, mappings, and rules: you import the library in SWIFT Translator Designer and add it to your project. As of then, you can start editing the predefined messages, mappings, and rules.



The details of this process and the customisation guidelines are explained further in the document.

Please follow these guidelines to ensure proper use of the library.

Note *It is strongly discouraged to override any format/mapping definitions from the library. This might create unexpected behavior as well as additional maintenance effort when upgrading to newer versions.*

Note *Overriding definitions breaks the warranty coming from Standards. SWIFT does not provide support on modified libraries.*

1.3 Who Does What?

For both the integration and the customisation, the customer could opt to have it done by **SWIFT Professional Services or through an existing Designer/Developer Seat contract**. This document is in a first stage a tutorial for those customers who intend to do this themselves:

- Customers that purchased a library (/libraries) that further need to be customised before actually using them in their application (/application landscape).

  They must have a SWIFT Translator Bundle contract (as add-ons licence).
- Customers that purchased a library (/libraries) the definitions and rules of which can as such directly by applied on their messages in their application (/application landscape).

  They must have at least a SWIFT Translator Runtime contract.

1.4 Available Libraries and Documentation

As of August 2020, the following libraries are available:

- **T2 ESMIG CLM Validation Library**

  This library contains the predefined message formats and validation rules of the new CLM ISO 20022 standard.

  CLM or Central Liquidity Management is new service that is introduced by the Eurosystem in the context of the T2-T2S consolidation, a major transformation of T2.

  For more information about this library and the scope of message definitions included, see the Libraries for SWIFT Translator T2 ESMIG CLM Validation Release Letter.
- **T2 ESMIG RTGS Validation Library**

  This library allows you to validate messages (the structure/format of messages) against the new ISO 20022 MX format of T2 (in the context of the new Eurosystem Single Market Infrastructure Gateway). For more information about this library and the scope of message definitions included, see the Libraries for SWIFT Translator T2 ESMIG RTGS Validation Release Letter.
- **T2 ESMIG RTGS Translation Library**

  This library allows you to translate messages from/to the MT format to/from the new ISO 20022 MX format of T2. For more information about this library and the scope of messages (/translations) included, see the Libraries for SWIFT Translator T2 ESMIG RTGS Translation Release Letter.

  These translation rules and the ISO 20022 messages are based on Target 2 's usage guidelines, UDFS version 2.1., published on https://www2.swift.com/mystandards/#/group/T2
- **CBPR+ Validation Library**

  This library allows you to validate messages (the structure/format of messages) against the CBPR+ ISO 20022 MX format. For more information about this library and the scope of message definitions included, see the Libraries for SWIFT Translator CBPR+ Validation Release Letter.
- **CBPR+ Translation Library**

  This library contains a set of predefined mappings from MT to the CBPR+ ISO 20022 format, and the other way around. These mappings correspond to the translation rules of the CBPR+ messages, as agreed within the CBPR+ working group. For more information about this library and the scope of messages (/translations) included, see the Libraries for SWIFT Translator CBPR+ Translation Release Letter.

  Additional documentation related to the CBPR+ initiative, the specifications, and the translation rules may be found on the Cross-border Payments and Reporting Plus landing page on MyStandards.

Translation portal on MyStandards (for CBPR+ only)

This portal allows you to check and assess the actual behavior of the CBPR+ libraries. It is a live and dynamic translation portal with up-to-date translation rules and message translation testing facilities: you can easily test the translation of individual messages from MT to MX and vice versa.

Furthermore, a set of translation documentation describing the translation rules can be downloaded from the portal homepage. This documentation is kept up-to-date.

The portal can be reached from the Cross-border Payments and Reporting Plus landing page on MyStandards.

Note *The access to such portal is controlled.*

2 Translation Library Details - Structure

2.1 Base Message

The Standards translation libraries are often built using "usage guidelines", like CBPR+. Messages that have usage guidelines variants can theoretically be parsed in two ways.

- Strict parsing where the usage guideline is merged with the base definition. A tag that is not allowed in the usage guideline will generate a syntax error.
- Base message parsing where the usage guidelines/restrictions are ignored.

The Standards translation libraries will always parse against the base message, even when a library is built in the context of a usage guideline. The translation is assuming that the message is following the usage guideline, but never enforcing it. If the consuming application cannot guarantee that the input is following the usage guidelines, **it is recommended to first validate the message using a Standards validation library.**

2.2 Mapping Hierarchy

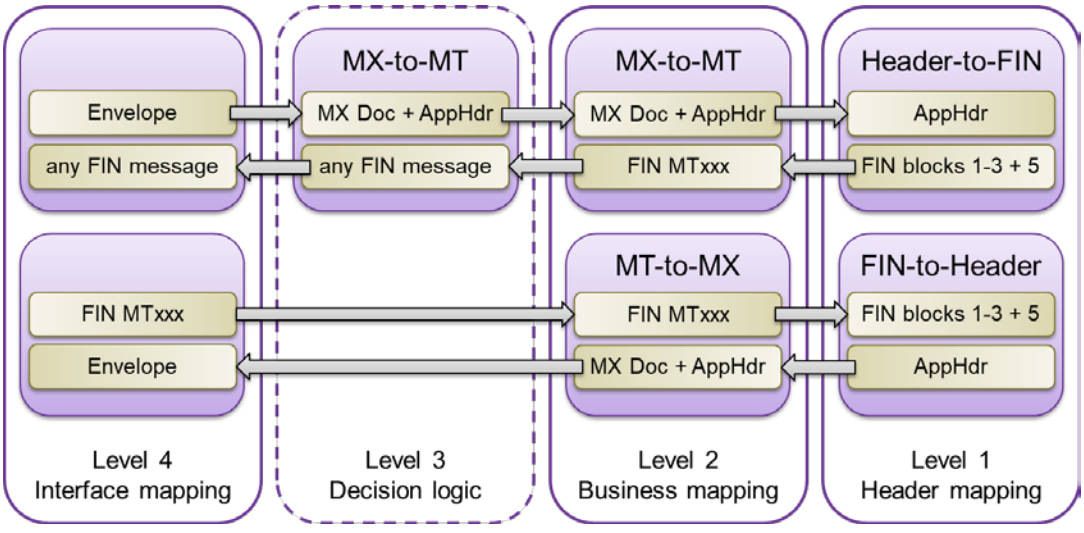
In SWIFT Translator all mappings are organised in a hierarchical way. As such, following types of mappings can be distinguished:

- Common mappings, like MX-to-MT header mapping
- Message content mappings
- Interface mappings, to be called by the consuming application to start the mapping

There are 4 mapping levels:

- Level 1 – Header mapping (and other common mappings like mapping of a financial institution)
- Level 2 – Business message mapping
- Level 3 – Decision logic (optional level)
- Level 4 – Interface mapping

It is important for the functioning and use of the translation libraries to understand and take into account these mapping levels, as illustrated below:



2.2.1 Level 1 - Header Mapping

The header mapping is a common mapping logic which is called for every message to either:

- build a FIN header from an MX business application header or
- build an MX business application header from a FIN header

The input for those mapping rules is limited to only the blocks of the message that are relevant:

- in case of MX-to-MT mapping: the input is the MX Business Application Header and message type
- in case of MT-to-MX mapping: the input is block 1, 2, 3, and 5 (in fact a trailer, but in this context considered as part of the "header" mapping) of the FIN message and the message application identifier

The output is similarly defined:

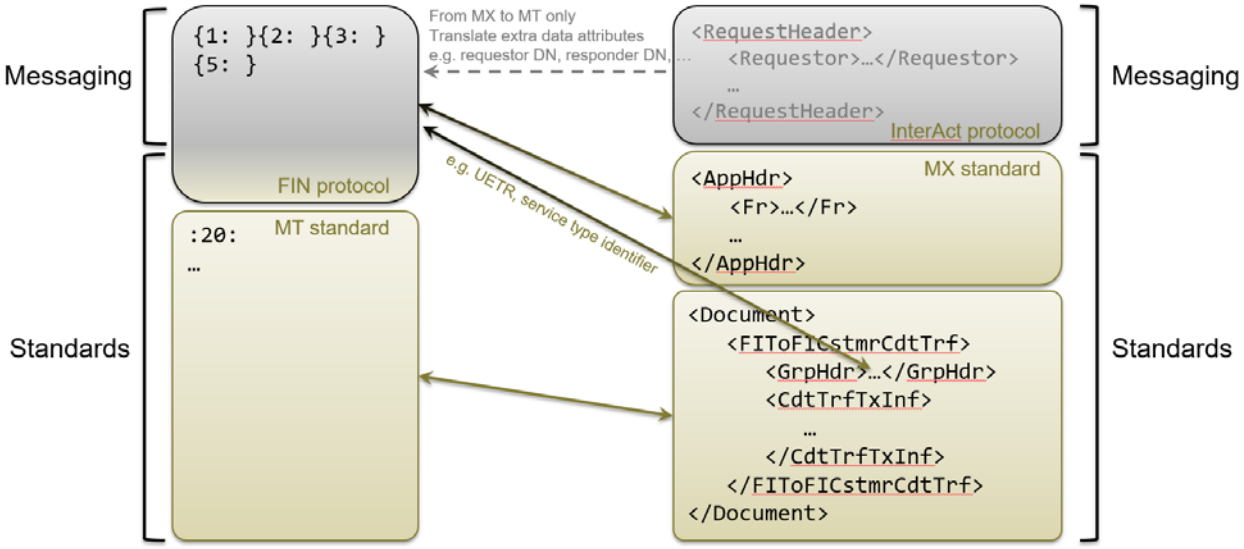
- in case of MX-to-MT mapping: the output is block 1, 2, 3, and 5 of the FIN message
- in case of MT-to-MX mapping: the output is the MX Business Application Header

From MX	To MT
Business Application Header (BAH) + Message Type	Blocks 1, 2, 3, and 5

From MT	To MX
Blocks 1, 2, 3, and 5 + Message Definition Identifier	Business Application Header (BAH)

2.2.1.1 No Perfect Mapping Between MT and MX Headers

When mapping MX to MT or MT to MX, these headers will not perfectly match as depicted here :



The FIN header blocks contain information that is not available in the MX Business Application Header. The FIN header is a mixture of transport specific data (for example, the input and output time) and business data.

The MX Business Application Header does not contain all data fields that are available in the FIN header: some data fields are matched with a field in the MX document itself. For example, the UETR.

For more information about these mappings rules, see the Libraries for SWIFT Translator T2 ESMIG RTGS Translation Release Letter and the Libraries for SWIFT Translator CBPR+ Translation Release Letter.

2.2.1.2 Additional Runtime Parameters for Mapping MX to MT ("FIN parameters")

In the MT format, the header blocks (1,2, 3, and 5) do contain fields that are not present in the Business Application Header of the MX. The translation libraries that map MX to MT provide default values for these fields (zero's, default date, ...).

For many mappings from MX to MT, extra FIN attributes (for example, transport specific data) can be specified to populate these fields in the FIN header (/override the default values). FIN attributes can also be specified to determine whether block 2 is input or output.

This can be accomplished via the so-called Runtime parameters: when invoking a function (see further), it is possible to specify extra parameters that can change the behaviour of such function. The mapping functions take additional Runtime parameters allowing you to add extra transport-specific data to the FIN header. These parameters will be injected into the FIN header blocks as is.

The following parameters can be specified in the Runtime context when calling a mapping and will be used to populate the FIN header:

Parameter	Format	Description	Example
Block3_433	/3!a/20x		
Block3_434	/3!a/20x		
Block3_165	/3!c/34x		
SendersAddress	AAAAAAAA or AAAAAAAABBB or AAAAAAAAXBBB	The SWIFT address of the sender, either as 8-character BIC, 11-character BIC, or logical terminal (LT). The value is completed with X's until a valid LT is reached. Depending on the application mode, this value will be added to block 1 or block 2.	BANKBEAAXYYY
ReceiversAddress	AAAAAAAA or AAAAAAAABBB or AAAAAAAAXBBB	The SWIFT address of the receiver, either as 8-character BIC, 11-character BIC, or logical terminal (LT). The value is completed with X's until a valid LT is reached. Depending on the application mode, this value will be added to block 1 or block 2.	BANKBEBBXZZZ
InputTime	HHmm	Time as hours/minutes, added in block 2 in case of output mode.	1245
OutputTime	HHmm	Time as hours/minutes, added in block 2 in case of output mode.	1246
ApplicationMode	I or O	Indicates whether block 2 should be an input block or output block, defaults to O.	

Parameter	Format	Description	Example
Block5_PDE	TRUE or FALSE	Possible duplicate indicator.	
Block5_TNG	TRUE or FALSE	Training indicator.	
MessagePriority	U or N	Urgent or Normal priority.	
InputDate	yyMMdd	Date as year/month/day, added in block 2 in case of output mode.	201213
OutputDate	yyMMdd	Date as year/month/day, added in block 2 in case of output mode.	201213

2.2.2 Level 2 - Business Message Mapping

The business message mappings contain the actual mapping logic for each combination of input and output message. The first line in such mapping is called the level 1 header mapping.

For more information about these business mappings, for CBPR+, see the CBPR+ Landing Page on MyStandards: https://www2.swift.com/mystandards/#/c/cbpr/landing.

For T2 ESMIG RTGS, see Mapping Specifications (.xls files) on the Download Centre.

From MX	To MT
Business Application Header (BAH) + ISO Document (for example, pacs.008.001.08)	Full FIN (Block 1 → 5)

From MT	To MX
Full FIN (Block 1 → 5)	Business Application Header (BAH) + ISO Document (for example, pacs.008.001.08)

2.2.3 Level 3 - Decision Logic

This mapping level is an optional mapping level that can be used when the same input message (message type) can be mapped on more than 1 output message (message type). Some examples:

- A pacs.009 can be mapped either onto an MT 202 (core), or to an MT 202 COV.
- A pacs.004 can be mapped either onto an MT 103 or an MT 202 (core).

In both examples it will be necessary to wrap the existing mappings with an output agnostic mapping. The output of this level 3 mapping will be `sys:anyNode` which links to the outcome of any level 2 mapping.

Just as illustration, the rule logic for pacs.009 could behave as follows:

From MX	To MT
IF the input pacs.009 contains an underlying transfer	Map the output (MT 202 COV) to the output of the decision mapping (`sys:anyNode`)

ELSE	Map the output (MT 202 CORE) to the output of the decision mapping (`sys:anyNode`)

The input is always the same as in the level 2 rules that are called.

2.2.4 Level 4 - Interface Mapping

The interface mapping is the last mapping level that wraps all input or output blocks in one common input or output block. It also allows multiple flavours (that is, different input messages) to map the same business message.

From MX	To MT
<Envelope>	Full MT

From MT	To MX
Full MT	<Envelope>

The interface mapping will always call the level 2 business message mapping or the level 3 decision logic. The common elements from the input message will be used as input for the lower level mappings. The output is defined as `sys:anyNode` which links to the output of the lower level mappings.

For more details on the mappings in the Standards translation libraries, see the Libraries for SWIFT Translator CBPR+ Translation Release Letter and the Libraries for SWIFT Translator T2 ESMIG RTGS Translation Release Letter.

3 HOW to Integrate the Libraries and Invoke the Functions Directly from your Application(s)

There are different ways for integrating the libraries and use them as they are delivered (that is, no customisation). The most common and easy way is the Message Validation and Translation API (MVT API). SWIFT provides this MVT API as an inherent part of the SWIFT Translator Runtime component. It is meant for the customers that aim to do the integration themselves. Therefore, this MVT API method is explained more in detail here.

Other integration options exist, but the implementation has to be done through SWIFT Professional Services. In some of these options, the SWIFT Translator Designer is also necessary as part of the integration process. Just for the sake of completeness, we list these options here, referring also to the respective specific technical documentation.

3.1 MVT API

3.1.1 Overview

The Message Validation and Translation API (MVT API) is a simplified Java API. It allows the user to read a .jar life and trigger its functions. Following functions can be accessed through the API.

- Parsing a message into an internal structural form (using a message definition)
- Executing a validation, checking whether an input message meets its definition and any validation rules
- Executing a mapping, translating one input message into one output message (multiple input or output is currently not supported)
- Evaluating a condition, checking an input message, and return true or false
- Executing enrichment, adding data to a message

As explained above, the libraries are delivered in the form of .jar files. Such .jar file contains all relevant message definitions, mapping definitions, condition definitions, validation rule sets, and enrichment rules sets.

In order to call/trigger a function, you need to use the unique key each definition or rules set is identified by. You will find those unique keys in the release letters of the respective libraries. The use of these unique keys and where to find them will be further detailed here, and illustrated with an example.

3.1.2 Integration – Prerequisites

Runtime libraries

The MVT API is a set of jars that must be added to the classpath of your application. You will need the following sets, which can be downloaded from the Download Centre.

- Translator Runtime: the SWIFT Translator Runtime component which includes the MVT API
- CurrencyLib: contains configuration files for currency tables and is delivered separately

Call back implementation for reference data check (only applicable for validation libraries)

The SWIFT validation libraries inherently come along with a facility to check/validate the following reference data: BIC, IBAN, country code, and currency code.

These checks are accomplished via a call back mechanism: The library will call back the consuming application. Therefore, you need to implement a call back class in the class path.

SWIFT provides a default implementation which generates "return true" on every such call back. You can use this one if you want to skip this reference data check. Your BIC, IBAN, … validation is inherently part of your Business As Usual (BAU) processing and you do not want those to additionally be checked during message format validation.

For this default implementation, you need the following java class path (additional .jar file), included in the validation library package on the Download Centre:

- SWIFTMX-CallBackDefinition.jar

Note *When the call back classes are not properly defined, the validation library integration will fail.*

3.1.3 Flow

The MVT API process to integrate a library and invoke its functions directly from your application(s) consist of the following 3 steps:

STEP 1: create a translator project with the delivered library

```
TranslatorProject project = new TranslatorProject(inputStream,
                new TranslatorXPathValidationErrorBuilder(true));
```

`inputStream` This represents a reference to the downloaded library file.

`TranslatorXPathValidationErrorBuilder` This is a validation error builder provided by the MVT API. It formats SWIFT Translator Runtime errors, which can be complex, into errors with an appropriate content for the user.

STEP 2: parse the input message and check for syntax errors

```
ParsedMessage parsedMessage =
                project.parseMessage(content, messageKey);
List<ValidationError> syntaxErrors =
                parsedMessage.getParsingErrors();
```

`content` The input message content as a string.

`messageKey` The unique key the message definition is identified with.

`parsedMessage` The message being parsed.

`syntaxErrors` The errors reported on the syntax (see further).

STEP 3: invoke the function

Execute a **Validation function**

```
List<ValidationError> validationErrors =
                project.applyValidationRuleSet(parsedMessage,
                                               validationKey);
```

Execute a **Mapping function**

```
ParsedMessage result = project.applyMapping(parsedMessage,
                                            mappingKey);
List<ValidationError> errors = result.getParsingErrors();
```

`validationKey` The unique key the validation rule set is identified with.

`validationErrors` The validation errors reported (see further).

Hereunder we will further elaborate some important elements of this integration process.

3.1.4 Project Information – Unique Keys to Invoke a Function

In order to invoke a function, you need the unique key of that function. An easy way to get these keys is to create an XML with general information on the library (that is, on the project you created in step 1). This general information will include all unique keys of the definitions and rulesets included in that library. This is illustrated here with an example, for the CBPR+ validation library:

```
TranslatorProjectInformation projectInformation =
                                    project.getProjectInformation();
String infoAsXML = projectInformation.toXMLString();
```

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<TranslatorProject>
    <name>CBPR_VAL</name>
    <version>1.1.4</version>
    <status>RELEASED</status>
    <description>CBPRplus Usage Guideline Definitions</description>
    <buildTime>2020-04-22T19:37:55</buildTime>
    <messageKeys>
        <messageKey>pacs.002.001.10</messageKey>
        <messageKey>camt.054.001.08</messageKey>
        <messageKey>...
    </messageKeys>
    <conditionKeys/>
    <mappingKeys/>
    <validationRuleSetKeys>
        <validationRuleSetKey>MX-Validation</validationRuleSetKey>
        <validationRuleSetKey>pacs.002.001.10-swift.cbprplus</validationRuleSetKey>
        <validationRuleSetKey>camt.052.001.08-swift.cbprplus</validationRuleSetKey>
        <validationRuleSetKey>...
    </validationRuleSetKeys>
    <enrichmentRuleSetKeys/>
</TranslatorProject>
```

3.1.5 The Message Parsing Function

This is always the first step before validating or translating a message. The input of a parsing function is plain text (content) which will be parsed against the message definition (identified by the *message key*).

The outcome will be a Java object of type `ParsedMessage`. This object represents the internal structure to be used by the other functions (validation, mapping, enrichment, or condition evaluation) in the next steps. If a message cannot be parsed, an exception `ParseException` is thrown.

The parsing function also validates syntactically the message. Raised syntax errors are listed in the `ParsedMessage` object and should be checked and reported to the end user. It is up to the calling application to assess the severity of the errors reported and what to do with them.

3.1.6 The Validation Function

The validation function can be triggered on a `ParsedMessage`, using a unique *validation rule set key* to identify one or several rule sets to apply.

The outcome of the validation is a list of errors. The way these errors are reported is determined by the validation error builder included in the MVT API (see step 1): `TranslatorXPathValidationErrorBuilder.`

The purpose of this error builder is to format internal SWIFT Translator Runtime errors, which can be complex, into errors with the appropriate content for the user. As such, internal errors are formatted to `ValidationErrors` that contain the following attributes:

- String getErrorCode()
- String getMessage()
- String getMainNodePath(): the location in the message
- List<String> getRelatedNodePaths(): optionally, multiple paths can be reported when an error is related to more elements in the message
- Optional<String> getNodeValue(): the value of the element, represented by the main node path

When a validation is not possible, an `ActionException` is thrown. This should not happen and can only occur when predefined assumptions have been broken.

The `ActionException` has multiple sub-classes.

3.1.7 The Mapping Function

The mapping function can be triggered on a `ParsedMessage`, using a unique *mapping key* to identify a mapping definition.

The outcome of the mapping is a `ParsedMessage`. It may contain errors in case the result is not syntactically valid against its definition, or, if the mapping definition adds errors in the output. These errors will help to assess the quality of the mapping and to potentially rectify.

When a mapping is not possible, an `ActionException` will be thrown. This may happen if one of the actions defined in the mapping could not be executed or if the purpose of the action is to raise an exception (for example, user defined pre-conditions are not met so the user wants to stop the mapping).

Mapping keys

As explained, the mapping keys are the unique keys to use when invoking a mapping function. You find the specific keys in the respective library release letter, or in the general project information (see above). In general, the mapping keys follow the following usage convention:

```
<message type>[-<usage context>]%<message type >[-<usage context>]
```

- The input and output message types are separated by %. A message type refers to a base message or message standard and can be supplemented by a usage context, referring to the usage guidelines in case of MX.

On the input side, the message type will always refer to a specific base message, whereas on the output side the message type can be generalised to a base message standard, like MT or MX. The latter is useful when an input message can result into multiple output messages. For example, a pacs.009.001.08 can result in an MT 202 or MT 202 COV (depending on the availability of an underlying transaction).

- `Message type`: mx, mt, fin.202.2019, fin.202.COV.2019, pacs.008.001.08, camt.053.001.08, ...
- `Usage context`: swift.cbprplus.01, trgt.t2.rtgs.01, ...

Examples

- fin.202.2019%mx-trgt.t2.rtgs.01:
  translation of an MT 202 to the corresponding MX message of Target2 (trgt.t2.rtgs.01)
- pacs.008.001.0-swift.cbprplus.01%mt:
  translation of a pacs.008.001.08 customer credit transfer of CBPR+ (swift.cbprplus.01) to the corresponding MT

3.1.8 Other Functions

As listed above, some other functions might be part of a library. This can be the case for libraries that you (or Professional Services) built from scratch in SWIFT Translator Designer. This is not applicable for the Standards libraries as such, but we mention them here for the sake of completeness.

Condition evaluation

The condition evaluation can be triggered on a `ParsedMessage`, using a unique *condition key* to identify a condition.

```
boolean result = project.evaluateCondition(parsedMessage, conditionKey);
```

The outcome of the condition evaluation is a boolean: "true" if the message meets the condition, "false" otherwise.

Enrichment

The enrichment function can be triggered on a `ParsedMessage`, using a *unique enrichment rule set key* to identify a rule set.

```
List<ValidationError> errors =
          project.applyEnrichmentRuleSet(parsedMessage, enrichmentKey);
```

The enrichment updates the parsed message and returns the list of syntax errors, if any.

3.1.9 Runtime Parameters

When invoking a function, it is possible to specify extra parameters that can change the behaviour of the function.

Examples

- See above, header mapping: The mapping functions take additional Runtime parameters allowing you for instance to add extra transport specific data to the FIN header when mapping from MX to MT.
- When evaluating conditions, a parameter can be specified to check against, for example, check whether an account occurs in a list of accounts.

The parameters can be added to the Runtime context before invoking the function in question.

For illustration, adding a sender's and receiver's BIC, a date, … to the header when MX to MT :

```
RuntimeContext ctx = new RuntimeContextImpl();
ctx.putUserObject("SendersAddress", "BANKBEAAYYY");
ctx.putUserObject("ReceiversAddress", "BANKBEBBZZZ");
ctx.putUserObject("InputDate", "201213");
...
ParsedMessage parsedMappedMessage = project.applyMapping(parsedMessage,
           "pacs.008.001.08-swift.cbprplus.01%mt", ctx);
```

3.1.10 Chaining Functions

With the MVT API, the client code orchestrates its message processing. After parsing an input message, the calling application decides what to do with the parsed message and it is able to react to the results of the different called functions.

It is possible to apply multiple functions on the same parsed message or to call another function on the result (parsed message) of a mapping.

A possible scenario is for instance:

1. Parse message
2. Validate message
3. Map message
4. Validate mapped message

Just for illustration, the complete sequence for this scenario:

```
String content = …
String messageKey = …
String inputValidationKey = …
String mappingKey = …
String outputValidationKey = …
Try {
    ParsedMessage parsedMessage = project.parseMessage(msgContent, messageKey);
    List<ValidationError> syntaxErrors = parsedMessage.getParsingErrors();
    if (syntaxErrors != null && syntaxErrors.size() > 0) {
        // The input message is not valid syntactically
    }

    List<ValidationError> validationErrors =
                project.applyValidationRuleSet(parsedMessage, inputValidationKey);
    if (validationErrors != null && validationErrors.size() > 0) {
        // The input message is invalid compared to the specified validation rules
    }

    ParsedMessage result = project.applyMapping(parsedMessage, mappingKey);
    List<ValidationError> errors = result.getParsingErrors();
    if (errors != null && errors.size() > 0) {
        // Either the translated message is not valid syntactically
        // or the mapping definition added errors
    }

    List<ValidationError> validationErrorsOut =
               project.applyValidationRuleSet(parsedMessage, outputValidationKey);
    if (validationErrorsOut != null && validationErrorsOut.size() > 0) {
        // The translated message is invalid compared to the specified validation
          rules
    }
}
```

3.2 Runtime API

For more information, developers should consult Runtime for SWIFT Translator Calling SWIFT Translator at Runtime.

3.3 Webservices

For more information, developers should consult Runtime for SWIFT Translator RESTful Webservices.

3.4 Maven

For more information, developers should consult Runtime for SWIFT Translator Maven.

3.5 Command Line Tools

For more information, developers should consult Runtime for SWIFT Translator Command Line Tools.

4 HOW to Customise a Library

When you want to add extra rules to your mappings, add/change message definitions, or change the behaviour partially, it is advised to customise the provided Standards library first. Before you actually start to translate your messages.

➔ We list some use cases here, and explain the high-level steps to follow.

4.1 Use Cases, Types of "Customisation"

Non-exhaustive list of use cases where it is necessary to customise the libraries first in the SWIFT Translator Designer:

- New translation/mapping for a complete new message type.

  For example, when a translation or a message definition is out of the scope of the translation/validation library.
- Extra field mappings.

  A camt.053 message for instance can contain much more information than an MT 940. You might want to map extra information from the camt.053 to the MT 940 or a separate structured message (csv, proprietary xml…).

  For example, the field Additional Statement Information in a camt.053 statement might contain relevant information that must be mapped to the outgoing MT 940. An MT 940 has a field :86: at the end of the message, which allows to capture this information.
- Change the output message type.

  For example, create a mapping to a proprietary message.
- Change the input message type.

  For example, you have your own format (FINX, csv, proprietary XML, Cobol Copybooks, etc.) which can be similar to FIN, but contains more/different information.
- Enrichment – add extra information that comes from an extra source.

  For example, instead of mapping an MT into ISO, you would like to use and extra customised CSV as input to map more information to the ISO.

Note *The above explained concept of Runtime parameters (for example, to populate some MT header fields when translating MX to MT) is not a customisation in this category. You do not need the SWIFT Translator Designer to add Runtime parameters when invoking library functions. This is performed "at runtime".*

4.2 HOW to Customise a SWIFT Standards Library?

As explained above, in SWIFT Translator Designer you can customise (change, amend, extend, ...) a Standards library.

➔ You need a SWIFT Translator Bundle (Designer + Runtime component) licence.

In order to do so, you first need to import the Standards library in SWIFT Translator Designer. Next, you can start customising the predefined messages, mappings, and rules and/or add additional logic around it.

Furthermore, you will find some general customisation guidelines. For more information, see the SWIFT Translator training received during the SWIFT Translator customer onboarding. As a licensed customer, you can also contact mystandards@SWIFT.com for support assistance.

Note *Before you can actually start validating and translating messages, based on the customised library, you need to deploy the Runtime component in your application/production environment. It is the SWIFT Translator project in which you imported and adapted the library that needs to be "deployed". This is the classical SWIFT Translator (Designer --> Runtime) workflow.*

4.2.1 How to Import a Library in SWIFT Translator Designer

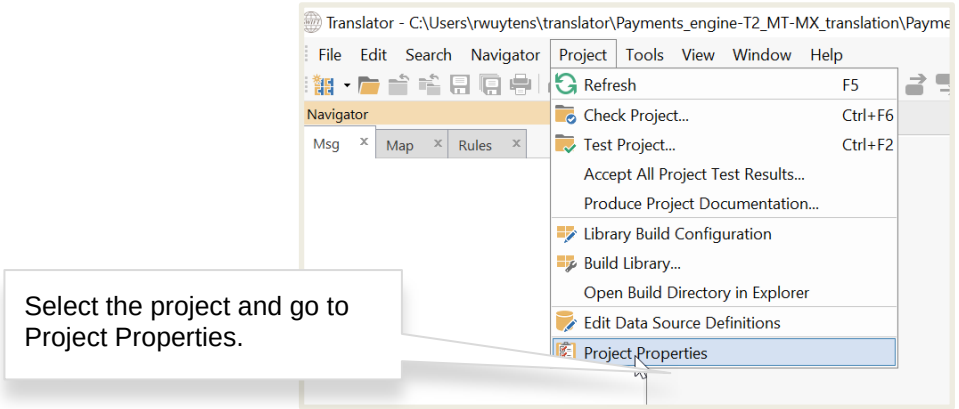
In summary, the steps to follow are:

1. Create a project in SWIFT Translator Designer.
2. Within that project, import and add the Standards library (.jar file).
3. Change/add message definitions, mapping definitions, and/or rule sets.

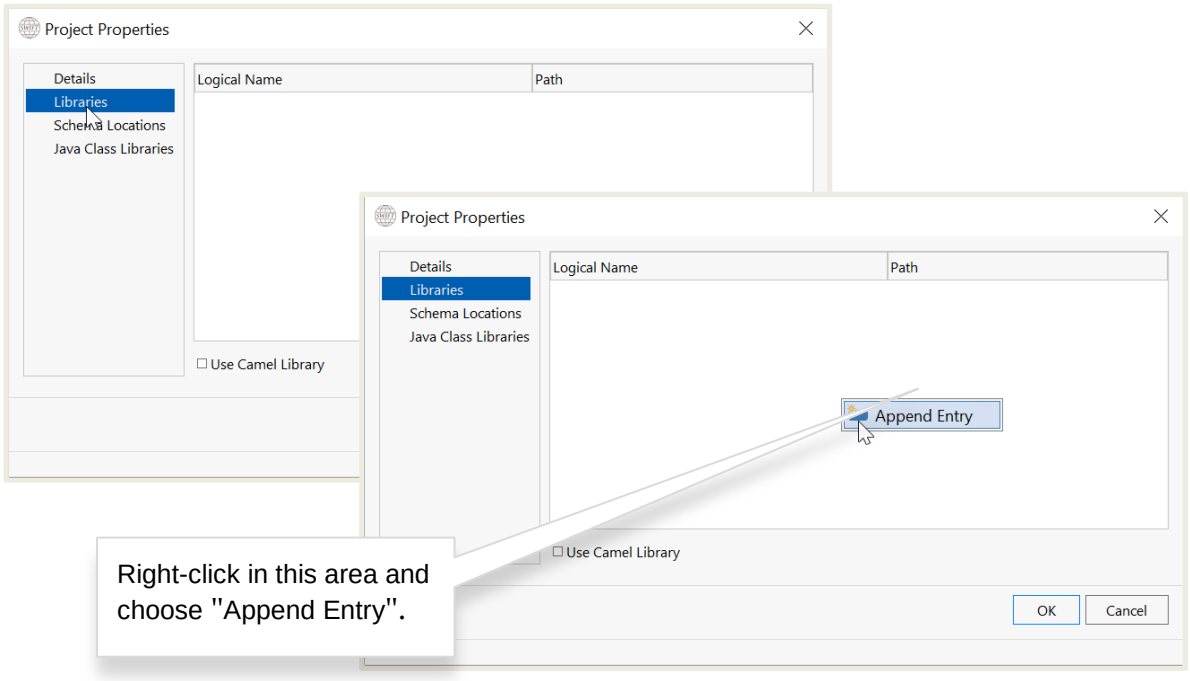
Example:

We want to add a particular mapping definition that is not supported in the T2 ESMIG RTGS translation library. We will import the library in a Designer project that we already created: "Payments_Engine-T2_MT-MX"

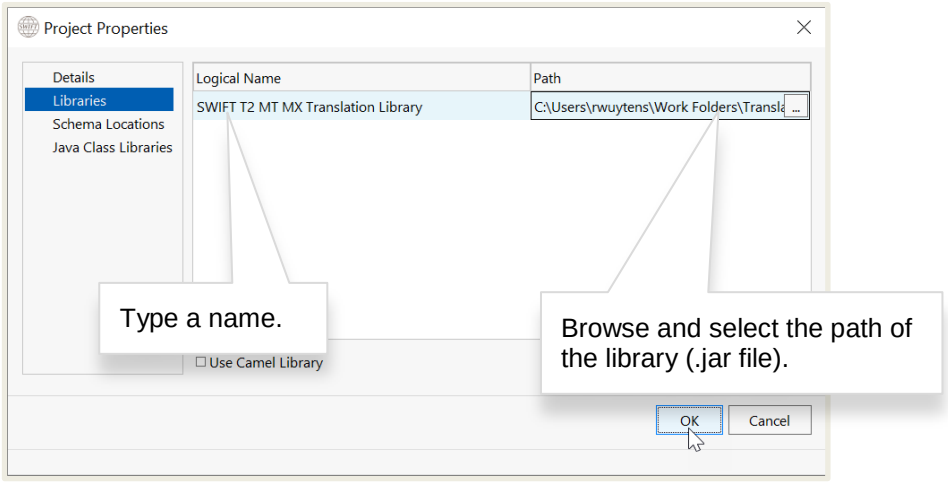
- Open the corresponding .tpj file (Payments_Engine-T2_MT-MX.tpj) in SWIFT Translator.
- Go to the project properties:



- Go to Libraries.

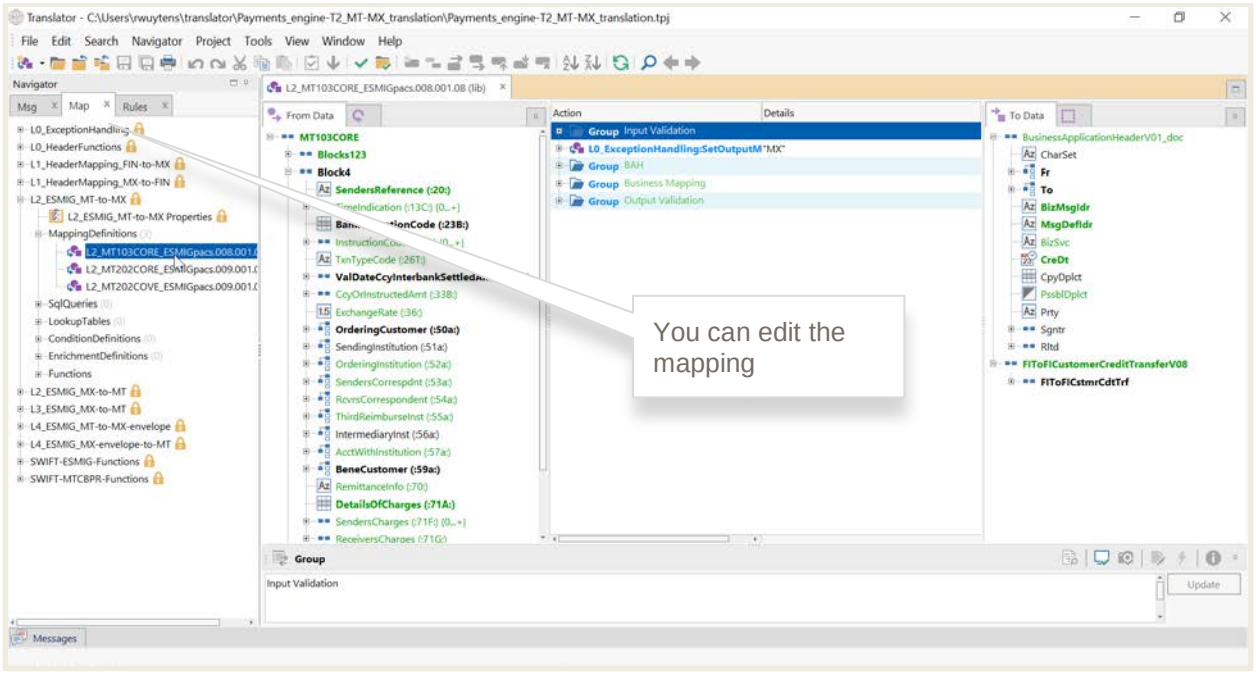


- Give a logical name, and indicate the path of the downloaded .jar file.



- You have now added the T2 ESMIG RTGS translation library to the "Payments_Engine-T2_MT-MX" project.

  All the message definitions, mapping definitions, validation rules, etc. are now visible in the navigator pane, under the respective "Msg", "Map", and "Rules" tabs. You can now add your missing mapping definition here :



4.2.2 General Guidelines

When editing a Standards library, it is important to follow some general principles. One must bear in mind the maintainability over the years, as new, upgraded versions of the Standards libraries will be released : Libraries are updated in line with the Standards releases (yearly for SWIFT MT).

Some basic guidelines:

- You should build further on the existing library structure, respecting the different mapping layers (in case of translation libraries).
- You should not override existing definitions. Create/add new definitions instead.
- Do not add definitions to existing definition groups. Add new definition groups.

Note *It is strongly discouraged to override any format/mapping definitions from the library. This might create unexpected behavior as well as additional maintenance effort when upgrading to newer versions.*

Note *Overriding definitions breaks the warranty from Standards. SWIFT does not provide support on modified libraries.*

5 Related Documentation

- SWIFT Translator Service Description
- Libraries for SWIFT Translator T2 ESMIG RTGS Validation Release Letter
- Libraries for SWIFT Translator T2 ESMIG RTGS Translation Release Letter
- Libraries for SWIFT Translator CBPR+ Validation Release Letter
- Libraries for SWIFT Translator CBPR+ Translation Release Letter
- CBPR+ Landing Page on SWIFT's MyStandards Platform: https://www2.swift.com/mystandards/#/c/cbpr/landing.
- Runtime for SWIFT Translator Message Validation and Translation API

Legal Notices

Copyright

SWIFT © 2020. All rights reserved.

Restricted Distribution

Do not distribute this publication outside your organisation unless your subscription or order expressly grants you that right, in which case ensure you comply with any other applicable conditions.

Disclaimer

The information in this publication may change from time to time. You must always refer to the latest available version.

Translations

The English version of SWIFT documentation is the only official and binding version.

Trademarks

SWIFT is the trade name of S.W.I.F.T. SC. The following are registered trademarks of SWIFT: 3SKey, Innotribe, MyStandards, Sibos, SWIFT, SWIFTNet, SWIFT Institute, the Standards Forum logo, the SWIFT logo and UETR. Other product, service, or company names in this publication are ademarks of their respective owners.